SIMATS ENGINEERING — CSA37: Software Testing

Assessment Tool 2 (AT2): Short Answer Test Solutions (Part 1: Q1–Q15)

Q1: Structured Test Design

Structured test design applies systematic techniques (BVA, Equivalence Partitioning) instead of random testing, ensuring higher defect detection and rigorous test coverage.

Q2: Standardized Banking Testing

A defined testing process in banking prevents systemic failures by introducing structured reviews, reducing defect leakage, and ensuring consistent transaction validation.

Q3: SDLC Process Discipline

Integrating testing across all SDLC phases ensures early bug detection in integration and system levels, lowering remediation costs and enforcing high quality.

Q4: Error, Defect, Failure

An error is human mistake, a defect is its manifestation in code, and a failure is the observable incorrect system output during execution.

Q5: Early Defect Detection Principle

"Early testing saves time and money." Catching requirements ambiguities early prevents costly downstream defects in later implementation phases.

Q6: Tester in Requirement Phase

Testers review requirements early to identify ambiguities and incomplete specifications, preventing downstream design and implementation flaws.

Q7: Requirement Clarity & Defect Origin

Unclear requirements spawn logical faults. Precise, clear requirements establish unambiguous baselines, preventing misinterpretations and logic errors.

Q8: Boundary Input Crashes

Classified as boundary/robustness defects. Boundary Value Analysis (BVA) tests limit values explicitly, preventing unexpected crashes from extreme inputs.

Q9: Defect Repository Utility

Repositories store historical bug data, guiding targeted test case design, highlighting error-prone modules, and preventing repeated release defects.

Q10: ATM Functional Defect

Transaction logic failure during withdrawal is a functional defect. Prevented via rigorous functional test cases and state transition verification.

Q11: Defect Clustering

Concentration of bugs in few modules (Pareto principle). Directs testing effort toward high-risk areas to optimize efficiency and find most defects early.

Q12: Performance Defect Classification

Slow web performance under load is classified as a non-functional (performance/load) defect, resolved via concurrency tuning and stress testing.

Q13: Regression Test Data Reuse

Reusing historical defect data in regression testing optimizes coverage, focuses on previously unstable areas, and improves execution efficiency.

Q14: Developer-Tester Communication

Early tester involvement and joint reviews bridge communication gaps, minimizing integration defects through aligned expectations.

Q15: Long-Term Defect Prevention

Defect prevention techniques (root cause analysis, process refinement) eliminate recurring systemic issues, enhancing overall software reliability.

SIMATS ENGINEERING — CSA37: Software Testing

Assessment Tool 2 (AT2): Short Answer Test Solutions (Part 2: Q16–Q30)

Q16: Mobile App Standardization

Standardizing testing principles and frameworks across mobile modules ensures consistent test execution and higher app reliability.

Q17: Testing vs. Debugging

Testing identifies failures and bugs; debugging locates and fixes the source code root cause. Both are vital for quality assurance.

Q18: Cross-Browser Strategy

Cross-browser testing principles validate rendering compatibility across browsers, handling layout and script execution differences effectively.

Q19: Validation Gaps in Logistics

Improper validation causes inconsistent logistics outputs. Early input validation testing ensures reliable data processing and system reliability.

Q20: Systematic Test Design

Replacing random testing with systematic engineering test design enhances defect discovery and guarantees structured, traceable test coverage.

Q21: Healthcare Data Consistency

Integration testing identifies root causes of data mismatches between modules, ensuring strict database consistency and data integrity.

Q22: SDLC Tester Collaboration

Early tester integration across all SDLC phases improves collaboration, catches requirement flaws, and elevates overall deliverable quality.

Q23: Peak Usage Failures

Untested peak usage scenarios cause crashes. Comprehensive load testing and edge-case coverage prevent production performance failures.

Q24: Defect Documentation

Proper defect documentation and repositories enhance tracking, streamline communication, and facilitate deep statistical failure analysis.

Q25: Banking Reliability Redesign

Redesigning testing approaches with structured methodologies and automation eliminates repeated transaction failures and builds reliability.

Q26: Edge Case Testing

Boundary testing incorporates edge cases beyond normal inputs, uncovering hidden vulnerabilities and boosting overall defect detection.

Q27: Process Standardization

Standardized process-based testing eliminates module-wise quality variations, ensuring consistent, repeatable software outcomes.

Q28: Robust Input Validation

Rigorous input validation testing handles unexpected user inputs correctly, ensuring application robustness and security against invalid data.

Q29: Production Defect Leakage

Root cause analysis of defect leakage identifies testing gaps, enabling improved pre-production test practices to prevent escapes.

Q30: Early Detection Cost Efficiency

Early defect detection strategies reduce overall project cost and effort by catching flaws when rectification is inexpensive and swift.